

2E Phase 2/3 Documentation

Design

Semantic Checking

On a high level the semantic checker takes in an AST as input and modifies it to produce a high-level intermediate representation as output. This modification is done in the following order:

1. Enforce order of operations within the AST by visiting each node recursively from the root, modifying the tree in place, and fixing the shape of the tree at that node based on operator precedence. All modification occurs exclusively within expressions.
2. Each node of this AST is visited to create the appropriate symbol table and entries for each node. To simplify our code, each AST node has its own, immutable symbol table. The symbol table for any given node stipulates what symbols would be available when it is first reached during runtime. Every scope has a new symbol table pointing to its parent's symbol table.
3. Semantic checks are performed according to the Decaf specification. For this part, we heavily use the visitor pattern, with each visitor roughly corresponding to a rule in the Decaf spec. During this step, the following checks are done:
 - a. Type checking is done using a type propagation visitor which returns the expected overall type of an expression. The expected type depends on, for example, whether it is called within another expression expecting a certain type (i.e. the plus operator expects integers) or an assign statement expecting a certain type for the assignee.
 - b. The symbol table is used to ensure variables and methods have been declared properly during this stage.
 - c. Return type checking is done separately from regular type checking.
 - d. Other rules are organized by purpose with multiple rules depending on the visited node type.
4. Since the checker visitor can output multiple errors for a given parent node, an error collector visitor is used to aggregate all of these errors and format them so they are easily readable.
5. Finally, after a program is semantically checked and has no errors, an intermediate AST representation and symbol table is returned, which can be used for control-flow graph creation and unoptimized code generation.

The Visitor Pattern

Our semantic checker heavily relies on the visitor pattern. This has allowed us to produce highly modular code that decouples the AST from the algorithms that check its semantics. We are already reaping the rewards of this design. For example, we had a mistake in which we were catching code without return statements at compile time instead of runtime. The fix involved commenting out the two lines of code that invoked the two corresponding visitors.

Immutable Tables

Each node in our AST has a single symbol table which is immutable. We did this to optimize for readability and ease of use. The design of our symbol tables can be thought of using two axes: the vertical axis corresponds to the nesting depth of a AST block (i.e. the block within an if statement or while loop, etc.), and the horizontal axis corresponds to sequences of statements. As the horizontal axis increases, the symbol tables will have more elements due to passing through declarations; as the vertical axis increases, there will be new symbol tables whose parents are those above them.

Error Collector

As visitors visit AST nodes, they deposit errors in a list-like data structure. When the semantic checking is finished, errors propagate up the tree and are selected to display meaningful error messages upon failure.

Code Generator

In overview, our unoptimized code generator takes in a high-level intermediate representation from the semantic checker, creates a low-level intermediate representation, then emits assembly code. This is done in the following steps:

1. Our code generator takes in the AST and IR, and generates basic blocks of each instruction for the control flow graph via a recursive destruct function that converts AST nodes to sequences of blocks based on their type. Our destruct function is largely inspired by the one discussed in class. For 'if', 'while', and 'for' statements, our short circuit function performs the modifications to the CFG for short circuiting conditionals during this step. Finally, our block maker visitor will generate either a forking block for statements with true and false execution paths or sequential blocks for statements in a linear order, which is determined by the AST node type. Once these are generated, blocks are merged together to form a connected control flow graph, without any expression flattening. When handling break, continue, and return statements, we had to make a design decision for whether to modify the control flow graph now to represent the different control flow or to deal with it when emitting assembly. We decided it was easier to modify the control flow graph now rather than try and emit specific code to deal with these statements.
2. After generating the control flow graph, we flatten instructions in it to get a structure that is more conducive for handling memory allocation. Our flatten visitor flattens expressions found in each block and allocates memory for each temporary variable in a register structure, allowing for easy organization and simplifying the translation from arrays in the IR to arrays in the emitted assembly. One major design decision we had to make was when to do flattening and memory allocation. We initially tried to do it as part of control flow graph generation, but reasoning about the generation code, flattening expressions, and memory allocation all in the same place was very cumbersome and hard to think about. As a result, we decided to separate CFG generation from expression flattening and initial memory allocation. This has the added benefit of making our code cleaner, easier to reason about, and increases modularity.

3. After the control flow graph is flattened and temporary variables are created, memory locations are assigned based on our initial allocations so that the variables are accessible via the stack. This is done through a memory allocator visitor which visits each statement in each block to allocate memory on the stack for declared variables. It also ensures that method call parameters and return values are assigned to the correct registers and locations. One issue we ran into during this stage was how to handle variables with the same name but in different scopes. We considered using a stack of tables that associate variables with their locations, but from previous experience we knew this would be difficult to work with. Instead, we associate each block with a table of variables that are within scope to make a tree of tables, and look up which memory location to use for the variable within the scope with this tree.
4. Finally, each instruction is visited in topological order so assembly code can be emitted, with accesses replaced by loads and stores corresponding to the location allocated within our memory table, alignment and initialization of strings and arrays, and runtime error handling added. Our runtime error handling raises different exceptions depending on the type of error. We decided to add this as it was part of the x86 specification, but none of the tests for this phase actually checked which exception was raised.

Extras

For memory allocation, we took some inspiration from LLVM and implemented registers as a way to hold values that instructions in blocks can read and write. While they are named registers, they can represent either actual registers or memory locations on the stack. We also implemented descriptive debugging outputs, which can print out control flow graphs for each method in a program, as well as how memory has been allocated for the program. We did not use any additional build scripts or external libraries.

Additionally, we had actually implemented ensuring that methods with a non-void return type actually had a return statement in them within our semantic checker, rather than implementing it as a runtime check; since phase 3 tests assumed it was done at runtime, we had to incorporate this runtime check during phase 3 despite having implemented it earlier.

The phase 3 tests also revealed a bug in our semantic checker that was not tested during phase 2: during phase 2, we assumed that the neither the location nor the expression for an increment or decrement assign operation could be an array location, so we raised an error if it was part of an array. A couple phase 3 tests revealed that this was not the case, so we had to revise parts of our semantic checker to account for this by differentiating between references to an entire array and references to an array element.

Difficulties

For semantic checking, we were passing 121/122 cases when we submitted the code by the deadline for phase 2. Just before starting on phase 3, however, we were able to implement mathematical order of operations properly so that we now pass every test case. As far as we are aware, there are no issues with our semantic checking implementation.

For code generation, we pass 50/50 test cases. After most of the code was written, we were failing many test cases due to minor bugs that affected multiple parts of various tests. After

hunting each of these down and fixing them, we were able to get each test case to pass. As far as we are aware, our code generator emits correct assembly code, but we may do refactoring in the future to make our work on phases 4 and 5 cleaner and more efficient.

Contribution

For the semantic checking portion of the project, we used Julius's parser for our base, and Julius and Adriano implemented most of the symbol table. All members worked together on implementing the various semantic checks, but the majority of the scaffolding was done by Julius and Adriano. Matt contributed to the initial type propagation checker and tests for it, as well as writing documentation. Noah implemented some of the output formatting and helpful refactoring. The only technical debt we had from this phase was not having mathematical order of operations implemented fully, but we were able to address this debt during phase 3.

For the code generation portion of the project, Adriano was responsible for the initial scaffolding for control-flow graph generation and part of the memory allocation logic, and this was then improved on by Julius to make the code more readable and eliminate unneeded methods and behavior. Both Noah and Julius extended this scaffolding: Julius was responsible for logic behind creating blocks, merging blocks, destructing, shortcircuiting, and for finalizing the API for control flow graph generation, and Noah was responsible for logic involving flattening the expressions in the control flow graph. Adriano was primarily responsible for memory allocation, with some help from Julius. Noah and Adriano were responsible for the logic behind emitting code after our compiler finishes all memory allocations, with further debugging done by all members of the team. Additionally, Matt debugged portions of the semantic checker after code generation tests revealed some small bugs that were not accounted for. Finally, Matt was primarily responsible for writing the documentation for this phase, with minor formatting and additional insight provided by Adriano.

Our main issues during phase 3 was maintaining our momentum through spring break and working in small chunks. The majority of our progress was not made until after spring break, and some of this progress would be in large blocks of code instead of gradual commits. For future phases, we aim to keep our momentum more consistent throughout the project, and ensure that code gets added in smaller chunks so that other members can understand the code easier and issues can be addressed as they come up naturally.